

2주차 공공 실시간 데이터는 어떻게 망가지는가

2주차 · 공공 데이터 품질

이 장의 수치는 모두 `practice/chapter2/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

다른 주차의 산출물을 가져다 쓴 곳에는 그 주차의 `practice/chapter{N}/data/output/` 경로를 함께 적었다.

- ☐ 1주차에서 데이터가 모델까지 가는 길을 봤다면, 이 장은 그 길에 실제로 흐르는 **공공 실시간 데이터**를 들여다봄
- ☐ 어떤 박자로 들어오는지, 어디가 비어 있는지, "값이 변했다"를 언제 문제로 볼지를 정함
- ☐ 이 장은 설명과 실습을 나누지 않음
- ☐ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
- ☐ 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

- ☐ "실시간"이라는 말은 항상 같은 속도를 뜻할까?
- ☐ 결측이 0건이라는 보고는 무엇을 세고 나온 말일까?
- ☐ 22시에 측정된 값을 22시 30분에 받았다면, 이것은 몇 시의 데이터일까?
- ☐ p-value가 크게 나왔다면 "두 시점의 분포가 같다"고 보고해도 될까?

2. 이 장에서 실행할 것

- ☐ 이 장의 네 주제는 우편물 처리소의 네 장면에 하나씩 대응함

우편물 처리소	실시간 데이터
편지가 들어오는 방식이 다르다 — 매일 조금씩, 명절에 폭증, 정해진 시간에 한 묶음	유입 패턴이 유형마다 다르다
소인 날짜(부친 날)와 도착 날짜는 다르다	이벤트 시간과 처리 시간
같은 편지가 두 번 오고, 봉투 규격이 말없이 바뀐다	중복과 스키마 변경
해가 바뀌며 오가는 우편물의 종류 구성이 달라진다	데이터 드리프트

표 2.1 우편물 처리소와 실시간 데이터의 대응

- ☐ 이 대응이 깨지는 지점이 하나 있음
- 우편물은 소인이 지워졌거나 봉투가 찢어진 것을 사람이 눈으로 봄
- 데이터에서는 값이 비어도 프로그램이 멈추지 않고 계산을 끝까지 마칩
- 그래서 무엇을 어떤 단위로 셀지 코드에 미리 적어 두어야 하며, 그 단위를 잘못 잡으면 무엇이 사라지는지가 3절에서 처음 숫자로 나타남
- ☐ 실행하기 전에 네 가지를 먼저 적음
- 산출물을 먼저 열면 거기 적힌 값이 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨

표 2.2 예측표 — 실행하기 전에 채운다

예측할 것	내 예측	실제	어긋났다면 원인
측정소 40곳의 <code>dateTime</code> 은 서로 몇 종류인가			
PM10 측정값(<code>pm10Value</code>)의 결측은 몇 건인가			
PM10 등급(<code>pm10Grade</code>)의 결측은 몇 건인가			
22:00과 23:00 비교에서 <code>drift_detected</code> 는 참인가 거짓인가			

- ☐ 이 장에서 실행할 코드는 둘이고 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	<code>code/2-1-parse-api.py</code>	응답을 표로 바꾸고 필드별 결측을 셈	3	<code>code/requirements.txt</code> 설치
2	<code>code/2-2-drift-visualize.py</code>	두 시각의 PM10 분포를 KS 검정으로 비교	6	위와 같음

- 둘 다 저장소에 커밋된 스냅샷 파일을 읽으므로 Docker도 인증키도 필요하지 않음
- 입력은 서울 40개 측정소의 실제 대기질 응답 두 장이며, 각각 22:00과 23:00 자료임

```
cd practice/chapter2
pip install -r code/requirements.txt
```

- `--live` 옵션은 이 장의 기준선을 바꾼다
- `2-1-parse-api.py --live` 는 인증키로 실시간 응답을 받아 EDA를 다시 계산한다. 그때의 대기질은 오늘의 값이므로 본문 수치와 대조할 수 없다.
- 1주차에서 인증키를 만들었고 수집 절차를 직접 보고 싶을 때만 쓰고, 본문과 대조할 때는 옵션 없이 실행한다.

3. 응답 한 장에서 확인할 것 — 들어오는 박자와 빈 칸

- 실시간 데이터라고 하면 초 단위로 쏟아지는 장면을 떠올리기 쉬움
- 공공 데이터는 그렇지 않고, 들어오는 방식이 네 갈래로 갈림

유입 패턴	어떻게 들어오는가	공공 예
지속 유입	업무시간 내내 대체로 일정한 속도	접수되는 민원
버스트	평소 조용하다 특정 사건에 폭증	재난 문자 발송 직후의 신고
고빈도	초 단위로 대량 유입	교통 센서 스트림
주기 갱신	정해진 간격으로 한 묶음씩	분·시 단위 환경 측정값

표 2.3 공공 실시간 데이터의 네 가지 유입 패턴

- 이 구분은 이름표를 붙이려는 것이 아니라 **설계를 미리 갈라 두려는 것이다**
- 평균 유입량에 맞춰 용량을 잡으면 버스트가 오는 날 시스템이 포화됨
- 반대로 한 시간마다 갱신되는 자료를 1분마다 호출하면 API 호출 한도만 소진되고 새 데이터는 없음
- 그러므로 호출 주기를 정하기 전에 **원천이 어느 패턴인지부터 응답으로 확인함**
- 확인하는 방법은 단순함
- 한 번의 응답에 들어온 레코드들의 측정 시각이 제각각이면 지속 유입이나 고빈도에 가까움
- 전부 같은 시각이면 원천이 한 묶음씩 갱신하고 있다는 뜻임
- 응답을 열면 두 번째로 셀 것이 결측임
- 공공 API는 값이 없을 때 필드를 빼지 않고 **"-" 같은 문자를 채워 보내는 경우가 많음**
- 이 표기를 그대로 숫자로 바꾸면 오류가 나거나, 더 나쁘게는 0으로 바뀜
- PM10이 0인 측정소는 "대기질 아주 좋음"으로 집계되어 경보 판단을 반대로 돌림
- 그래서 파싱 첫 단계에서 결측 표기를 결측으로 바꾸고, 바꾼 뒤 **필드마다 따로 셸**
- 첫 번째 코드를 실행함. 이 절의 숫자는 전부 이 명령의 출력임

```
cd practice/chapter2
python3 code/2-1-parse-api.py
```

```
# 출처: practice/chapter2/results/2-1-parse-api.log
source=file:airquality_seoul_2200.json stations=40 data_time=2026-06-28 22:00 pm10_missing=0
```

```
[pm10_summary] {"min": 14.0, "max": 27.0, "mean": 20.025, "median": 20.0}
```

```
[pm10_grade_counts] {"1": 28, "2": 11, "nan": 1}
```

[측정 시각] 서로 다른 dateTime 1개 - 2026-06-28 22:00

[결측 대조] 필드별 결측 건수 (결측 표기: None / '' / '-')

pm10Value 결측 0건 / 40건

pm25Value 결측 0건 / 40건

o3Value 결측 0건 / 40건

no2Value 결측 0건 / 40건

so2Value 결측 0건 / 40건

coValue 결측 0건 / 40건

khaiValue 결측 1건 / 40건

pm10Grade 결측 1건 / 40건

pm25Grade 결측 0건 / 40건

khaiGrade 결측 1건 / 40건

[값은 있는데 등급이 없는 측정소] 1곳 - 강남구

강남구 pm10Value='21' pm10Grade=None khaiValue='-'

□ 화면에 나온 값에서 읽을 것

○ **측정 시각이 한 종류임** — 40개 측정소가 전부 2026-06-28 22:00 을 달고 왔음. 이 원천은 측정소마다 따로 밀어 보내는 스트림이 아니라 정시마다 한 묶음을 갱신하는 **주기 갱신형**이며, 표 2.3의 네 갈래 중 어디인지가 응답 한 장으로 정해짐

○ **측정값 여섯 필드는 결측 0건임** — 여기까지만 보고 "결측 없음"이라고 쓰면 보고가 틀림

○ **같은 응답에서 pm10Grade 는 1건이 비어 있음** — 값을 세는 칸과 등급을 세는 칸의 결측 건수가 다름. 결측률을 데이터 전체 하나의 숫자로 내면 이 차이가 평균에 묻힘

○ **비어 있는 곳은 강남구 한 곳이고, 그 측정소의 pm10Value 는 '21' 임** — 측정은 됐는데 그 값에서 계산되는 등급만 없음. 통합지수 khaiValue 도 같은 레코드에서 '-' 로 왔음

○ **pm10_grade_counts 의 세 번째 항목이 "nan": 1 임** — 등급 분포를 셀 때 결측을 버리지 않고 한 칸으로 남겨 두었기 때문임. 버렸다면 39개소만 세고도 합이 맞아 보여 결측을 놓쳤을 것임

- **결측 0건은 "센 것이 0건"이라는 뜻이다**

- 이 실행이 센 것은 MEASURE_COLS 에 적힌 여섯 필드뿐이다. 등급·플래그·통합지수는 그 목록에 없다.

- 품질 보고서에 "결측 0건"을 적을 때는 **무엇을 썼는지 함께 적는다**. 세지 않은 필드는 결측이 없는 것이 아니라 확인되지 않은 것이다.

4. 이벤트 시간과 처리 시간이 같으면 통계가 같린다

- **이벤트 시간**은 사건이 일어나거나 측정된 시각이다
- **처리 시간**은 시스템이 그 데이터를 받은 시각이다
- 둘은 거의 항상 벌어짐 — 네트워크 지연, 재전송, 원천의 게시 지연 때문임
- 3절의 실행에서 이 간극이 이미 나타났음
- 응답이 달고 온 `dateTime` 은 2026-06-28 22:00 이지만, 그 응답을 받아 파싱한 시각은 그보다 뒤임
- 그러므로 "이것이 몇 시 데이터인가"는 내 컴퓨터의 시계가 아니라 **응답 안의 `dateTime` 값**으로 답함
- 처리 시간으로 집계하면 "언제 일어난 일인가"가 흐려짐
- 구청이 22:00~23:00에 접수한 민원 40건 중 15건이 시스템 부하로 23시 이후에 적재됐다고 하자
 - 처리 시간 기준 집계는 "22시대 25건"으로 보고함
 - 다음 배치가 그 수치로 인력을 정하면 22시대는 실제보다 적게, 23시대는 많이 배치되어 두 시간대가 모두 어긋남
- 이벤트 시간 기준이었다면 40건이 온전히 22시대로 귀속됐을 것임
- 그런데 이벤트 시간으로 세기로 정하면 새 질문이 하나 생김
- 22시대 집계를 언제 마감할 것인가 — 늦게 오는 이벤트를 언제까지 기다릴 것인가
- **워터마크**는 이 기다림의 한계를 정한 시각 임계값이다
- 도착이 워터마크 안이면 수용해 이미 낸 집계를 갱신함
- 넘으면 폐기하고 폐기 건수만 올림

이벤트 시간과 처리 시간의 간극 — 워터마크(watermark) 판정

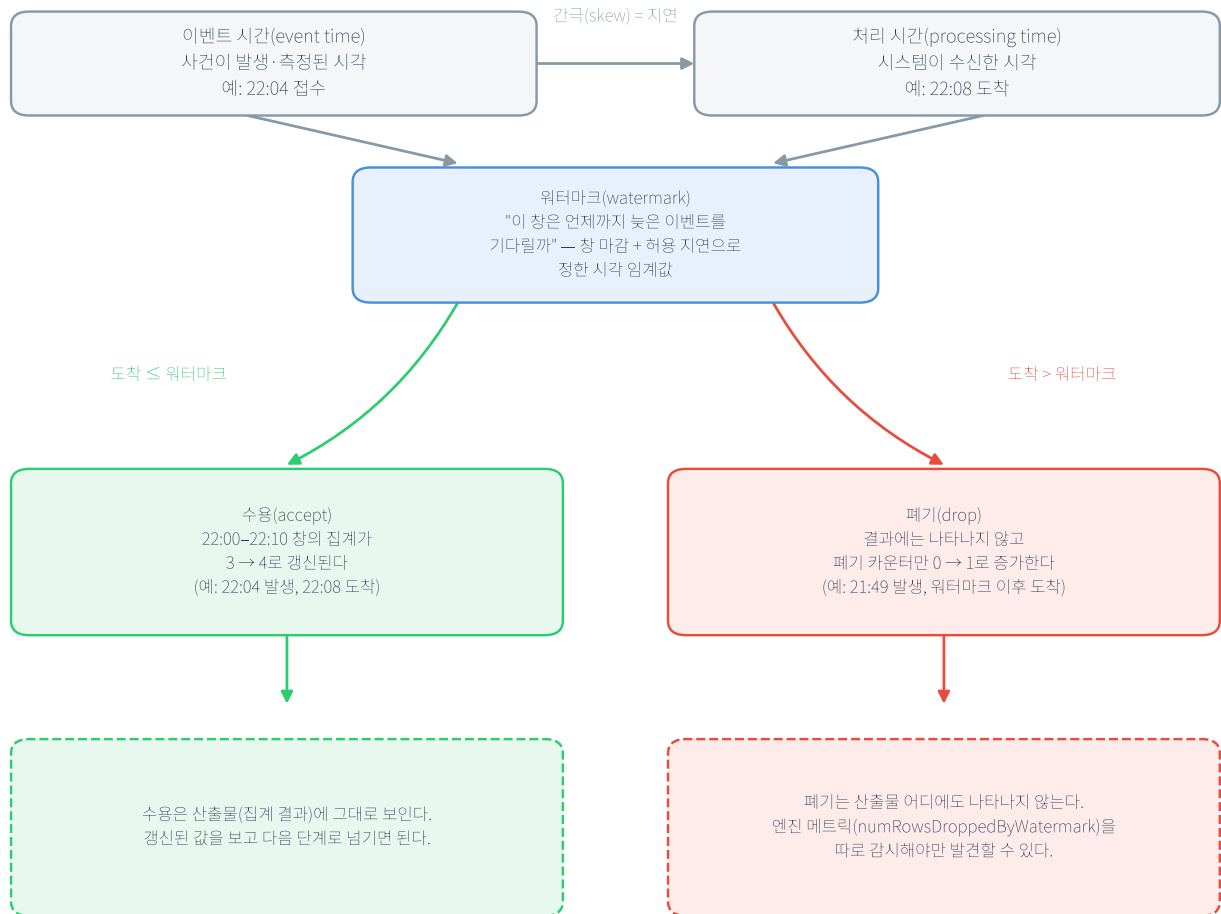


그림 2.1 이벤트 시간과 처리 시간의 간극, 그리고 워터마크(watermark) 판정

- 이 장의 실습은 측정 시각이 하나뿐인 스냅샷이라 지연을 만들어 볼 수 없음
- 수용과 폐기가 실제로 갈리는 장면은 5주차 실습이 산출물로 남겨 두었음

```

# 출처: practice/chapter5/data/output/ch5_watermark_trace.json
{ "file": "batch3.json", "events": 3, "watermark_in_effect": "22:08",
  "accepted": 2, "dropped": 1, "dropped_cumulative": 1 }

{ "event_id": "LATE_OK", "event_time": "22:04", "decision": "수용",
  "evidence": "창 22:00–22:10 강남구 count 3 → 4" }

{ "event_id": "LATE_DROP", "event_time": "21:49", "decision": "폐기",
  "evidence": "폐기 누적 = 1" }
  
```

- 화면에 나온 값에서 읽을 것
- 같은 "늦은 도착"인데 판정이 갈림 — 22:04에 일어난 이벤트는 수용되어 집계를 3에서 4로 갱신했고, 21:49에 일어난 이벤트는 워터마크 22:08을 넘겨 폐기됐음
- 수용은 결과에 보이고 폐기는 보이지 않음 — 수용된 건은 count가 4로 바뀐 것으로 확인되지만, 폐기된

건은 집계 어디에도 자국을 남기지 않음

- 그래서 폐기 건수를 따로 셈 — `dropped_cumulative` 가 1로 올라간 이 한 줄이 없으면 사라진 줄도 모름
- 원칙은 하나임
- 집계와 드리프트 판단은 이벤트 시간 기준으로 설계하고, 어떤 시간을 썼는지 산출물에 적어 둔다.

5. 중복과 스키마 변경은 오류 없이 지나간다

- 실시간 데이터에서 품질 문제는 예외가 아니라 정상 상태에 가까움
- 4절의 자연 말고도 반복해서 나타나는 것이 둘 더 있고, 둘 다 프로그램을 멈추지 않음
- 중복은 안전장치의 부작용이다
- 같은 이벤트가 두 번 이상 들어오는 일은 대부분 버그가 아니라 유실을 막으려는 재시도의 결과임
- 처리를 먼저 하고 기록을 나중에 하면 중간에 죽었을 때 같은 것을 다시 처리하게 됨
- 반대로 기록을 먼저 하면 중복은 없어지지만 처리 못 한 이벤트가 사라짐
- 민원 1건이 두 번 세어지면 "민원이 늘었다"는 잘못된 신호가 대시보드와 모델 입력에 그대로 들어감

```
# 출처: practice/chapter4/data/output/ch4_delivery_report.json
"B_at_most_once_crash": { "produced": 30, "processed_records": 10,
                          "unique_events": 10, "duplicated_events": 0, "lost_events": 20 },
"C_at_least_once_crash": { "produced": 30, "processed_records": 40,
                          "unique_events": 30, "duplicated_events": 10, "lost_events": 0 }
```

- 화면에 나온 값에서 읽을 것
- 같은 30건을 보내고 같은 지점에서 죽었는데 결과가 같림 — 한쪽은 20건을 잃었고 다른 쪽은 10건이 겹쳤음
- 같린 이유는 기록 시점 한 줄임 — 4주차에서 그 한 줄을 직접 바꿔 봄
- 중복을 막는 방법은 이벤트마다 고유 식별자를 붙이는 것임 — 같은 식별자가 다시 오면 처리하지 않고 흡수함
- 스키마 변경은 죽지 않고 조용히 틀린다
- 필드가 추가·삭제되거나 코드 체계가 바뀌어도 파서는 오류를 내지 않음
- 없는 필드를 찾으면 예외 대신 빈 값을 돌려줄 뿐이고, 집계는 계속 돌면서 결측률만 조용히 100%가 됨
- 1주차에서 본 침묵 실패가 여기서 다시 나타남
- 막는 방법은 표준 코드에 없는 표기를 억지로 끼워 맞추지 않는 것임
- 비슷해 보이는 자치구로 추정해 채우면 그 추정이 어디에 들어갔는지 나중에 되짚을 수 없음
- 대신 미매핑으로 분리해 세고, 들어온 건수와 나간 건수의 합이 맞는지 검사함

```
# 출처: practice/chapter6/data/output/ch6_batch_run_report.json (logical_date 2026-07-01)
"quality": { "physical": 20, "valid": 20, "mapped": 18, "unmapped": 1, "missing": 1,
             "unmapped_rate": 0.05, "alias_fixed": 4, "ws_stripped": 1,
             "preservation_ok": true, "gate": "PASS" }
```

□ 화면에 나온 값에서 읽을 것

○ **미매핑 1건이 숫자로 남음** — 표준 코드 사전에 없던 표기를 채워 넣지 않고 `unmapped` 라는 제 칸에 세워 두었음

○ **건수가 보존됨** — 매핑 18 + 미매핑 1 + 결측 1 = 유효 20으로 합이 맞고, `preservation_ok` 가 참임. 합이 어긋나면 배치 자체를 실패시켜 조용한 손실을 막음

○ **정리한 흔적도 함께 셈** — 별칭 교정 4건과 공백 제거 1건이 따로 기록됨. 무엇을 고쳤는지 세어 두어야 나중에 원천이 바뀐 것인지 규칙이 바뀐 것인지 가릴 수 있음

□ 성격이 다른 위험이 하나 더 있음

○ 민원 텍스트처럼 사람이 입력한 데이터에는 식별 가능한 값이 섞여 들어옴

○ 수집한 뒤에 지우려 하면 이미 로그와 백업에 퍼진 뒤라 되돌릴 수 없음

○ 그래서 마스킹은 사후 처리가 아니라 **수집 단계의 설계 조건**이며, 접근 통제는 12주차와 13주차에서 다룸

6. 값이 변한 것과 분포가 변한 것은 다르다

□ **데이터 드리프트**는 입력 데이터의 분포가 시간에 따라 변하는 현상이다

○ 모델은 학습 당시의 분포를 전제로 동작하므로, 입력 분포가 달라지면 코드가 한 줄도 바뀌지 않아도 성능이 떨어짐

○ 테스트는 코드를 고쳐야 깨지지만 드리프트는 아무도 아무것도 고치지 않아도 됨

□ 공공 데이터에서 드리프트를 부르는 원인은 네 갈래임

○ **정책 변화**로 민원 유형 코드가 개편되면 분포가 바뀜

○ **계절성**은 미세먼지·한파 민원처럼 매년 반복됨

○ **외부 이벤트**는 대형 재난처럼 일시적으로 폭증시킴

○ **시스템 변경**은 센서 교체나 API 변경으로 값의 범위와 결측 패턴을 바꿈

□ 여기서 바로 반대 방향의 주의가 붙음

○ **감지했다고 항상 경보를 울리면 안 된다.**

○ 계절처럼 매년 반복되는 변화를 매년 "이상"으로 경보하면 알림에 무뎌져 진짜 경보까지 무시하게 됨

□ 한 가지 구분을 덧붙임

○ 데이터 드리프트는 입력 분포의 변화임

○ **개념 드리프트**는 입력과 정답 사이 관계 자체의 변화임

- 같은 민원 문장이라도 "긴급"의 기준이 바뀌면 입력은 그대로인데 정답이 달라짐

- 변화를 재는 방법으로 **콜모고로프-스미르노프(Kolmogorov-Smirnov, KS) 2표본 검정**을 씀
- 두 표본의 누적분포 사이 최대 간격을 재고(통계량 D), "두 표본이 같은 분포에서 나왔다"고 가정했을 때 그만큼 큰 간격이 우연히 나올 확률(p-value)을 돌려줌
- 두 번째 코드를 실행함. 22:00 스냅샷을 기준으로 23:00 스냅샷을 비교함

```
cd practice/chapter2
python3 code/2-2-drift-visualize.py \
  --baseline data/input/airquality_seoul_2200.json \
  --current data/input/airquality_seoul_current.json \
  --col pm10Value
```

```
# 출처: practice/chapter2/results/2-2-drift-visualize.log
col=pm10Value base(2026-06-28 22:00,n=40,mean=20.025) curr(2026-06-28 23:00,n=40,mean=20.775) KS=0.175
p=0.579 drift=False
```

```
구간[14-16) 기준 ##### 5 현재 *****
2
구간[16-17) 기준 ##### 6 현재 *****
2
구간[17-19) 기준 ### 1 현재 *****
5
구간[19-20) 기준 ##### 10 현재
***** 12
구간[20-22) 기준 ##### 11 현재 *****
8
구간[22-24) 기준 0 현재 *****
3
구간[24-25) 기준 ##### 5 현재 *****
3
구간[25-27) 기준 ##### 2 현재 *****
5

분위수 기준 현재
0% 14.0 14.0
25% 17.0 19.0
50% 20.0 20.0
75% 22.0 23.0
100% 27.0 27.0
```

□ 화면에 나온 값에서 읽을 것

○ **값은 분명히 변했음** — 평균이 20.025에서 20.775로 올랐고, 25% 분위수는 17.0에서 19.0으로, 75% 분위수는 22.0에서 23.0으로 이동했음

○ **그런데 판정은 drift=False 임** — D가 0.175이고 p가 0.579여서 유의수준 0.05를 넘지 못했음

○ **양 끝은 그대로임** — 최솟값 14.0과 최댓값 27.0이 두 시각에서 같음. 가운데가 조금 밀렸을 뿐 분포의 폭은 움직이지 않았음

○ **중앙값도 20.0에서 20.0으로 같음** — 평균이 오른 것은 중간 구간의 측정소 몇 곳이 한 칸씩 옮겨 간 결과임

□ 이 결과를 어떻게 서술하느냐가 이 절의 핵심임

□ **첫째, p-value는 "분포가 변했을 확률"이 아니다.**

○ 두 분포가 같다고 가정했을 때 이만큼의 차이가 우연히 나올 확률임

○ 그러므로 p가 크다고 "두 분포가 같다"가 증명되지 않고, 정확한 진술은 **"**다르다고 볼 충분한 증거가 없다**"**이다

□ **둘째, 결과는 표본 크기에 좌우된다.**

○ 표본이 크면 사소한 차이도 유의해지고, 작으면 진짜 변화도 놓침

○ 이 실습의 표본은 측정소 40개로 작은 편이므로 작은 변화를 애초에 잡아내기 어려움

○ 산출물의 `baseline_n` 이 40인 것을 확인하지 않고 `drift_detected: false` 만 옮겨 적으면 검정력의 한계가 보고서에서 사라짐

□ 두 주의를 합치면 원칙이 하나 나옴

○ **단일 지표만으로 판정하지 않는다.**

□ 같은 기준 스냅샷에 9일 뒤 자료를 붙인 비교가 11주차 산출물에 있음

```
# 출처: practice/chapter11/data/output/ch11_drift_report.json
"comparison_a_matches_ch2": true,
"comparison_b_9days_time_aligned": {
  "baseline_time": "2026-06-28 22:00", "baseline_mean": 20.025,
  "current_time": "2026-07-07 22:00", "current_mean": 17.75,
  "ks_statistic": 0.225, "p_value": 0.2657, "ks_significant": false,
  "psi": 0.3324, "psi_band": "경보", "alert_level": "주의" }
```

□ 화면에 나온 값에서 읽을 것

○ **2주차 실측이 그대로 재현됐음** — `comparison_a_matches_ch2` 가 참인 것은 같은 스냅샷 쌍에서 KS 0.175와 p 0.5786이 다시 나왔다는 뜻임

○ **9일 뒤에는 평균이 20.025에서 17.75로 내려갔음** — 1시간 간격보다 훨씬 큰 이동임

○ **그런데도 KS는 여전히 유의하지 않음** — p가 0.2657로 0.05를 넘김. 반면 이동의 크기를 재는 PSI는 0.3324로 경보 대역에 들어감

○ **두 지표가 다른 답을 냄** — "이동의 크기는 경보 수준인데 이 표본 크기에서는 우연을 배제할 수 없다"가 이 상태의 정확한 서술이며, 지표 하나만 보면 이 문장을 쓸 수 없음

- 두 시각의 값이 다르다고 곧바로 드리프트가 아니다
- 대기질은 하루 안에서도 오르내리므로 22:00과 14:00을 비교하면 "시간이 흐른 변화"와 "시각이 다른 변화"가 섞인다.
- 11주차가 9일 뒤 비교를 굳이 같은 22:00으로 맞추는 이유가 이것이다. 비교창을 맞추지 않은 값은 신호가 아니라 교란이다.

7. 핵심 요약

□ 1. 결측은 데이터 전체가 아니라 필드별로 센다. (3절)

- 같은 응답에서 측정값 여섯 필드는 결측 0건인데 `pm10Grade` 는 1건이 비어 있었고, 비어 있는 강남구의 `pm10Value` 는 '21' 로 멀쩡했음
- 등급 분포를 셀 때 결측을 버리지 않고 "nan": 1 로 남겼기에 이 차이가 보였음
- "결측 0건"을 적을 때는 무엇을 썼는지 함께 적음

□ 2. 원천의 유입 패턴은 응답으로 확인하고 호출 주기를 거기에 맞춘다. (3절)

- 측정소 40곳이 전부 2026-06-28 22:00 이라는 한 종류의 `dateTime` 을 달고 온 것이 주기 갱신형이라는 증거임
- 평균에 맞춰 잡은 용량은 버스트가 오는 날 포화되고, 주기 갱신 원천을 자주 호출하면 호출 한도만 소진됨

□ 3. 집계 기준은 이벤트 시간이고, 폐기 건수는 따로 센다. (4절)

- 처리 시간으로 세면 22시대 민원 40건이 25건으로 보고되어 두 시간대의 인력 배치가 모두 어긋남
- 5주차 실측에서 22:04 이벤트는 수용되어 집계를 3에서 4로 갱신했고 21:49 이벤트는 폐기됐는데, 폐기는 집계 어디에도 나타나지 않아 `dropped_cumulative` 로 따로 세야 보임

□ 4. 중복과 스키마 변경은 오류 없이 지나간다. (5절)

- 4주차 실측에서 같은 30건이 기록 시점 한 줄 차이로 유실 20건과 중복 10건으로 갈렸음
- 6주차 실측에서 표준 코드에 없던 표기는 미매핑 1건으로 분리됐고, 매핑 18 + 미매핑 1 + 결측 1 = 유효 20으로 건수가 보존됐음
- 추정해 채우는 대신 제 칸에 세워 두고 합을 검사하는 것이 침묵 실패를 막는 방법임

□ 5. p-value가 크다고 "분포가 같다"가 아니다. (6절)

- 22:00과 23:00 비교에서 평균은 20.025에서 20.775로 올랐지만 D는 0.175, p는 0.579여서 판정은 `drift=False` 였음
- 정확한 서술은 "다르다고 볼 충분한 증거가 없다"이며, 측정소 40개라는 표본 크기가 그 판정의 조건임
- 9일 간격 비교에서 PSI는 0.3324로 경보인데 KS는 p 0.2657로 유의하지 않았으므로, 지표 하나로 판정하지 않음

8. 과제

☐ 실습을 순서대로 실행해 만든 산출물 두 개를 LMS에 그대로 업로드한다

1. `practice/chapter2/data/output/ch2_eda_summary.json`
2. `practice/chapter2/data/output/ch2_drift_result.json`

☐ 이어서 세 가지를 각각 세 문장 이내로 쓴다

1. 표 2.2에 적은 네 예측과 실제 값을 나란히 적고, 어긋난 항목마다 원인을 쓴다.
2. 본인 산출물의 `missing_by_field`와 `pm10_grade_counts`를 근거로, "이 응답에는 결측이 없다"는 보고가 왜 부정확한지 쓴다.
3. 본인의 `p_value`·`alpha`·`baseline_n`을 적고, `drift_detected`가 `false`인 것을 "두 시점의 분포가 같음이 확인되었다"로 보고하면 왜 틀린 서술인지 쓴다.

☐ 값을 지어내지 않는다

☐ 실행이 안 되면 어디서 막혔는지 그대로 쓴다

☐ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

☐ 실습을 더 해 보려면 `practice/chapter2/`에서 다음을 시도함

○ `2-2-drift-visualize.py`에 `--col pm25Value`를 주어 같은 두 시각을 PM2.5로 비교하고, PM10과 판정이 같은지 봄

○ `--alpha 0.2`로 유의수준을 올려 `drift_detected`가 뒤집히는지 확인하고, 임계값을 바꿔 판정을 뒤집는 것이 왜 분석이 아닌지 생각해 봄

○ `2-1-parse-api.py`의 `MEASURE_COLS`에 `khaiValue`를 넣어, 결측 집계 범위를 넓히면 보고가 어떻게 달라지는지 봄

○ `src/drift_detector.py`에는 KS 말고 PSI 구현도 들어 있음. 다만 `run_chapter2.py`가 쓰는 표본은 실측이 아니라 생성한 자료이므로 본문 수치와 대조하지 않음

☐ 뒤 주차에서 이어지는 내용

○ 중복과 전달 보증은 4주차, 워터마크와 창 집계는 5주차, 미매핑 분리와 배치 품질 게이트는 6주차에서 직접 실행함

○ PSI를 포함한 드리프트 판정과 재학습 절차는 11주차, 개인정보와 접근 통제는 12주차·13주차에서 다룸

☐ 원문을 직접 보고 싶다면 두 가지가 출발점임

○ SciPy, 「`scipy.stats.ks_2samp`」 — 이 실습이 쓰는 KS 2표본 검정의 계산 규약: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html

○ 실습이 호출하는 원천은 `2-1-parse-api.py`의 `AIRKOREA_ENDPOINT` 상수에 적혀 있으며, `--live` 없이 실행하면 저장된 스냅샷만 읽음